AIBuildAI: An AI agent that automatically builds AI models

Ruiyi Zhang* **Peijia Qin*** **Qi Cao*** **Li Zhang*** **Pengtao Xie†**
University of California, San Diego

1 Introduction

Building AI models remains a highly iterative and time-consuming process. Even for relatively well-scoped supervised learning tasks, producing a strong solution often requires substantial effort across environment setup, dataset preparation, exploratory analysis, feature design, model selection, implementation, debugging, hyperparameter tuning, and repeated evaluation [7]. While recent advances in large language models (LLMs) have enabled systems capable of code generation and tool use [4], building automated methods that can reliably execute the full end-to-end AI development workflow remains a challenging open problem [3]. A growing body of work has explored agentic frameworks in which LLMs interact with external tools, write code, and iteratively refine solutions [8]. However, many existing approaches rely on a single monolithic agent, which must maintain the full experimental history and intermediate artifacts of long-horizon AI workflows within a limited context window [6]. As a result, such systems often suffer from context overflow and information loss. In contrast, real-world AI research and development is typically performed by teams that divide responsibilities across different roles, such as designing candidate approaches, implementing models, tuning hyperparameters, and revising strategies based on empirical feedback. This motivates the use of an agent with multiple sub-agents as a natural structure for automating complex AI pipelines [5].

In this work, we present AIBuildAI, an AI agent that automatically builds AI models, with the goal of solving general AI tasks in an end-to-end manner. Motivated by the workflow of human AI researchers, AIBuildAI decomposes the AI development pipeline into a set of specialized sub-agents, each responsible for a distinct stage of the process. The workflow begins with a setup agent that initializes the environment and prepares the dataset. A central manager agent then orchestrates the overall pipeline, acting as a state machine controller that selects which agent to execute next based on the current progress and available artifacts. Typically, the manager first invokes a designer agent to propose multiple candidate modeling strategies, followed by coder agents that implement these strategies in parallel. The resulting candidate solutions are evaluated by a judge agent, which filters out weaker approaches, while tuner agents refine promising candidates through hyperparameter optimization on a validation split. Based on the observed performance, the manager may further invoke reviser agents to analyze outcomes and generate revised plans that guide the next iteration of experimentation, or terminate early if satisfactory performance has been achieved. Finally, an aggregator agent combines multiple strong candidate models to construct an effective ensemble strategy and outputs the final model and inference pipeline for direct user deployment.

We evaluate AIBuildAI on MLE-Bench, a benchmark of realistic Kaggle-style prediction tasks spanning visual, textual, time-series and tabular modalities and requiring end-to-end model development [3]. Without task-specific customization, AIBuildAI achieves the top position on the leaderboard with a medal rate of 63.1%, demonstrating performance on MLE-Bench comparable to highly experienced human AI researchers. These results highlight the effectiveness of structured multi-agent collaboration, where responsibilities are divided across specialized roles, candidate solutions are explored in parallel, and performance is improved through iterative evaluation-driven refinement. Overall, AIBuildAI provides a scalable framework that resembles the workflow of human

*Equal contribution
†Corresponding author: `p1xie@ucsd.edu`

Preprint.

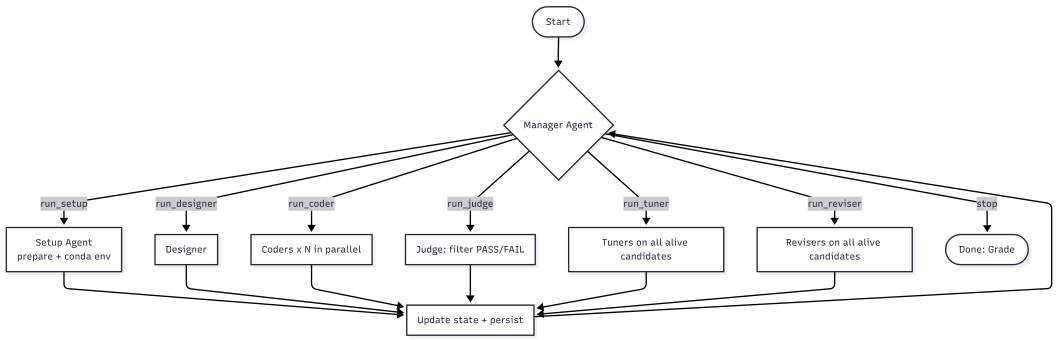

Figure 1: **AIBuildAI manager workflow.** A read-only manager agent emits a structured decision at each iteration, selecting a single next action (`setup`, `designer`, `coder(s)`, `judge`, `tuner(s)`, `reviser(s)`, or `stop`). Sub-agents write validated artifacts to disk, which are persisted into the workflow state and provided to the manager in the next iteration.

AI researchers and suggests that multi-agent architectures can move beyond isolated code generation toward fully autonomous AI development pipelines.

2 Method

2.1 System Overview

AIBuildAI automates end-to-end AI developing process by decomposing the workflow into specialized agents coordinated by a central *manager* agent. AIBuildAI follows the iterative loop common in human AI development practice: propose candidate approaches, implement and validate them, evaluate outcomes, tune promising solutions, and revise plans based on empirical feedback. AIBuildAI instantiates this loop as a manager-driven state machine that repeatedly (i) reads the current workflow state and on-disk artifacts, (ii) selects the next action, and (iii) invokes the appropriate sub-agent(s) to produce new artifacts and metrics.

AIBuildAI uses eight specialized agents:

- **Manager** (read-only access): reads the full workflow state and artifacts, and outputs a single decision specifying the next action and any additional instructions for sub-agents.
- **Setup**: prepares the dataset, initializes the environment, and records paths and dependencies.
- **Designer**: proposes multiple candidate modeling plans (e.g., feature pipelines, architectures, validation strategy).
- **Coder** (parallel, one per candidate): implements a plan into runnable code, performs a short smoke test, and produces a valid prediction file for the AI task.
- **Judge**: evaluates candidates at the current stage and filters weaker candidates.
- **Tuner** (parallel, one per surviving candidate): performs configuration-only hyperparameter tuning on the validation dataset and runs full training.
- **Reviser** (parallel, one per surviving candidate): analyzes artifacts and metrics and writes a revised plan for the coder to implement in a subsequent iteration.
- **Aggregator**: combines multiple strong candidate models via a soft-voting ensemble tuned on the validation split, and outputs the final ensemble model for submission.

2.2 Communication and Memory of Agents

AIBuildAI is designed around an explicit *artifact-based communication* protocol, where agents coordinate only through persistent files on disk. Instead of directly exchanging messages or sharing a global in-memory state, each agent is executed as a stateless subprocess that reads a fixed set of input artifacts and produces new output artifacts (JSON files, logs, metrics, and code). This design makes the workflow transparent, reproducible, and robust for long-horizon AI development.

Structured artifacts. All agent outputs follow predefined JSON schemas that serve as explicit interface contracts between agents. By enforcing structured, machine-readable communication rather than free-form text exchange, AIBuildAI reduces brittleness and enables deterministic downstream processing. In addition to task-specific outputs, each agent also produces a compact feedback record (e.g., summary, blockers, suggestions, and confidence), which is stored in the workflow history and later consumed by the manager for decision-making.

Persistent workflow memory. Because agents do not retain memory across invocations, the workflow history itself functions as the shared memory of the system. AIBuildAI persists all intermediate artifacts, evaluation results, and feedback traces to disk, allowing subsequent agents to reason over accumulated experimental evidence. This persistent memory mechanism is critical for scaling AIBuildAI to iterative AI workflows that involve multiple training runs, evaluation cycles, and refinement loops without requiring the full interaction history to fit within a single context window.

2.3 Manager Agent-Based Workflow Orchestration

AIBuildAI is orchestrated by a Python control loop that repeatedly invokes the manager agent and then executes the manager's chosen action. The manager observes the complete experiment history (actions taken, success/failure validations, metrics, per-step costs, and agent feedback) and uses this information to decide which stage to run next. The orchestrator serves as the execution backbone: it manages artifact I/O, enforces stage dependencies, and ensures that outputs produced by one agent are reliably consumed by subsequent agents.

Action space. The manager chooses from a fixed set of actions:

$$\{\texttt{run_setup}, \texttt{run_designer}, \texttt{run_coder}, \texttt{run_judge}, \texttt{run_tuner}, \texttt{run_reviser}, \texttt{stop}\}.$$

The orchestrator enforces basic preconditions (e.g., setup must precede designer; revisions require prior tuning history) and terminates the iterative loop when `stop` is selected or a global call budget is reached. After termination, AIBuildAI invokes an *aggregator* module that combines multiple strong candidate models via a soft-voting ensemble tuned on the validation split and outputs the final ensemble model for submission.

Plan evolution and revision tracking. AIBuildAI explicitly tracks plan evolution through versioned design artifacts. The initial plan produced by the designer is labeled as version 0, and each reviser update increments the version identifier. When a coder agent is invoked, the orchestrator always loads the most recent revised plan artifact if available, ensuring that implementation remains synchronized with the latest plan and that plan updates remain traceable throughout the workflow.

Instruction routing. AIBuildAI supports two complementary instruction channels for coordinating agent behavior. First, user-provided high-level strategy text can be injected into the manager prompt to guide global workflow preferences (e.g., emphasizing tuning or revision). Second, at each iteration the manager generates explicit instructions that are prepended to the prompt of the selected sub-agent. This mechanism enables dynamic coordination and targeted control while preserving fixed role definitions for each agent type.

2.4 Parallel Execution of Coder, Tuner, and Reviser

In AIBuildAI, the `coder`, `tuner`, and `reviser` actions execute in parallel over the current set of active candidates. Each candidate maintains an isolated working directory containing its own source code, configuration files, model checkpoints, evaluation metrics, and structured logs. This directory-level isolation prevents file conflicts and enables multiple candidate solutions to be developed and refined concurrently.

Parallelism is critical for accelerating the overall workflow, since both coding and hyperparameter tuning are typically time-consuming stages that involve repeated training runs. By allowing different agents to operate independently on different candidate plans, AIBuildAI can explore multiple modeling directions simultaneously, improving overall throughput and solution diversity. Within each candidate, the workflow remains sequential (i.e., $\texttt{coder}_i \rightarrow \texttt{tuner}_i \rightarrow \texttt{reviser}_i$), ensuring that later stages build upon the most recent implementation and experimental results. However, across

candidates, these stages are parallelized (e.g., $\texttt{coder}_i$ runs concurrently with $\texttt{coder}_j$), enabling efficient scaling with the number of candidate solutions.

If a subset of candidates fail during a parallel step due to runtime errors or invalid outputs, the workflow does not terminate immediately. Instead, the manager can continue with the remaining successful candidates, improving robustness and preventing isolated failures from blocking overall progress.

2.5 Judge Agent for In-Loop Filtering

During the iterative workflow, the manager may invoke the judge agent to filter candidate solutions and update the active candidate set. This filtering mechanism is essential for improving efficiency, since running multiple full training and tuning cycles for every candidate can be prohibitively expensive. By identifying invalid or low-quality candidates early, AIBuildAI can avoid wasting compute on unpromising directions and allocate resources to candidates that are more likely to succeed.

The judge operates in three stage-specific modes:

- **Post-designer**: assesses plan quality and feasibility before implementation, filtering out designs that are overly complex, ill-posed, or incompatible with the task constraints.
- **Post-coder**: checks implementation correctness by inspecting code structure and running a short smoke test to ensure that the training and inference pipeline executes successfully and produces a valid prediction file.
- **Post-tuner**: evaluates training logs and reported metrics to detect unstable optimization behavior, severe overfitting, or consistently underperforming candidates.

These stage-specific checks are motivated by the fact that different failure modes occur at different points in the AI workflow: a candidate may have a flawed design, a buggy implementation, or a training procedure that fails to converge. The judge outputs a `PASS`/`FAIL` decision per candidate along with rubric-based rationales, enabling the manager to prune candidates dynamically throughout the workflow. This in-loop filtering significantly reduces redundant computation and accelerates overall convergence by focusing tuning and revision effort only on viable candidates.

2.6 Safety, Logging, and Traceability

AIBuildAI applies hook-based sandboxing to reduce accidental corruption of datasets and to constrain sub-agents to safe operations. Write/edit tools are restricted to candidate-specific output directories, the prepared dataset directory is treated as read-only, and potentially destructive shell operations are blocked. Package installation is prohibited for all agents except the setup agent, which is explicitly permitted to install dependencies required for the task. These restrictions help ensure that iterative refinement improves solutions rather than destabilizing the environment.

AIBuildAI writes a comprehensive execution trace to disk, including (i) the full prompt sent to each agent, (ii) a streamed execution log capturing tool calls and outputs, (iii) structured JSON artifacts for every agent output, and (iv) an aggregated manager state file that summarizes the full workflow history. This design supports post-hoc inspection, debugging, and reproducible analysis of complete agent trajectories.

To improve auditability and reproducibility, the manager's decision is recorded in the on-disk history *before* sub-agent execution. After sub-agent execution, the history entry is updated with validation results, newly produced artifacts, refreshed metrics, and the measured cost and runtime of the step. This ensures that both decisions and outcomes are persistently captured, enabling transparent inspection and deterministic replay of the full workflow.

3 Results

Experimental Setup. For all experiments, AIBuildAI is powered by *Claude 4.5 Sonnet* as the underlying LLM [2], with each specialized role (e.g., setup, designer, coder, judge, tuner, reviser) implemented as an individual *Claude agent* [1]. We evaluate our method on OpenAI's MLE-Bench [3], which comprises a diverse set of tasks spanning multiple data modalities and problem formulations,

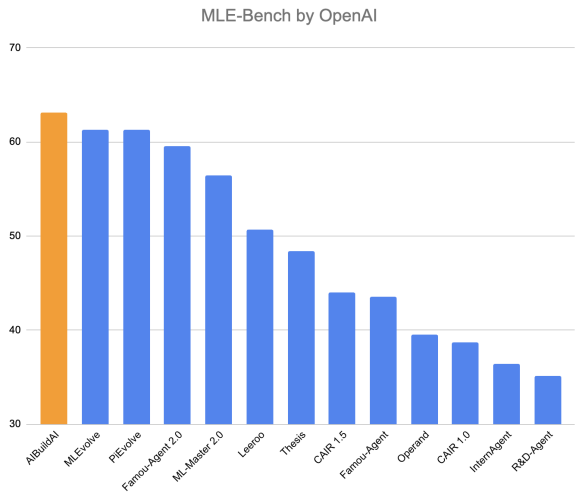

Figure 2: **AIBuildAI ranked #1 on OpenAI MLE-Bench.** We evaluate AIBuildAI on MLE-Bench, a benchmark of realistic Kaggle-style prediction tasks spanning visual, textual, time-series, and tabular modalities, requiring end-to-end model development [3]. Without task-specific customization, AIBuildAI achieves the top position on the March 6, 2026 leaderboard with a medal rate of 63.1%, demonstrating performance on MLE-Bench comparable to that of highly experienced human AI researchers.

including visual recognition, natural language processing, temporal forecasting, and structured tabular prediction. We evaluate AIBuildAI on 75 tasks from MLE-Bench covering these four major modalities. Each task provides raw datasets, evaluation metrics, and submission protocols similar to real-world AI competitions, requiring systems to design modeling strategies, implement training pipelines, and iteratively improve performance under limited computational budgets. Performance is measured using a medal system derived from human Kaggle competition rankings, where submissions are categorized as gold, silver, or bronze based on percentile thresholds achieved by human participants. The MLE-Bench grading protocol defines a deterministic public/private data split for Kaggle competitions. During execution, agents only have access to the public portion of the dataset, including training data, unlabeled test data, and the sample submission format, while the private ground-truth labels remain hidden. Upon completion of the AIBuildAI workflow, final predictions on the test set are evaluated against the hidden labels using the competition's official evaluation metric, and the orchestrator records the resulting score and corresponding medal status.

Integrity Checks. To ensure evaluation validity, at the end of each workflow run we apply an automated detector to inspect the full execution trace and verify that AIBuildAI does not directly copy online Kaggle solutions. We additionally verify that the workflow does not rely on external LLM services beyond the specified model. These checks help ensure that reported results reflect the performance of the autonomous workflow itself rather than leakage from external codebases or unauthorized model access.

Results. Figure 2 shows the performance of different methods on the MLE-Bench leaderboard as of March 6, 2026. AIBuildAI achieves the top overall position on the benchmark with a medal rate of 63.1%, ranking #1 on the leaderboard and demonstrating the effectiveness of the AIBuildAI strategy. In addition, AIBuildAI shows performance comparable to that of highly experienced human AI researchers.

Figure 3 gives an example of searching trajectory of AIBuildAI on the *Detecting Insults in Social Commentary* task in MLE-Bench. It further illustrates representative manager-driven execution trajectories, showing how AIBuildAI interleaves judging with parallel candidate refinement before terminating and triggering objective grading.

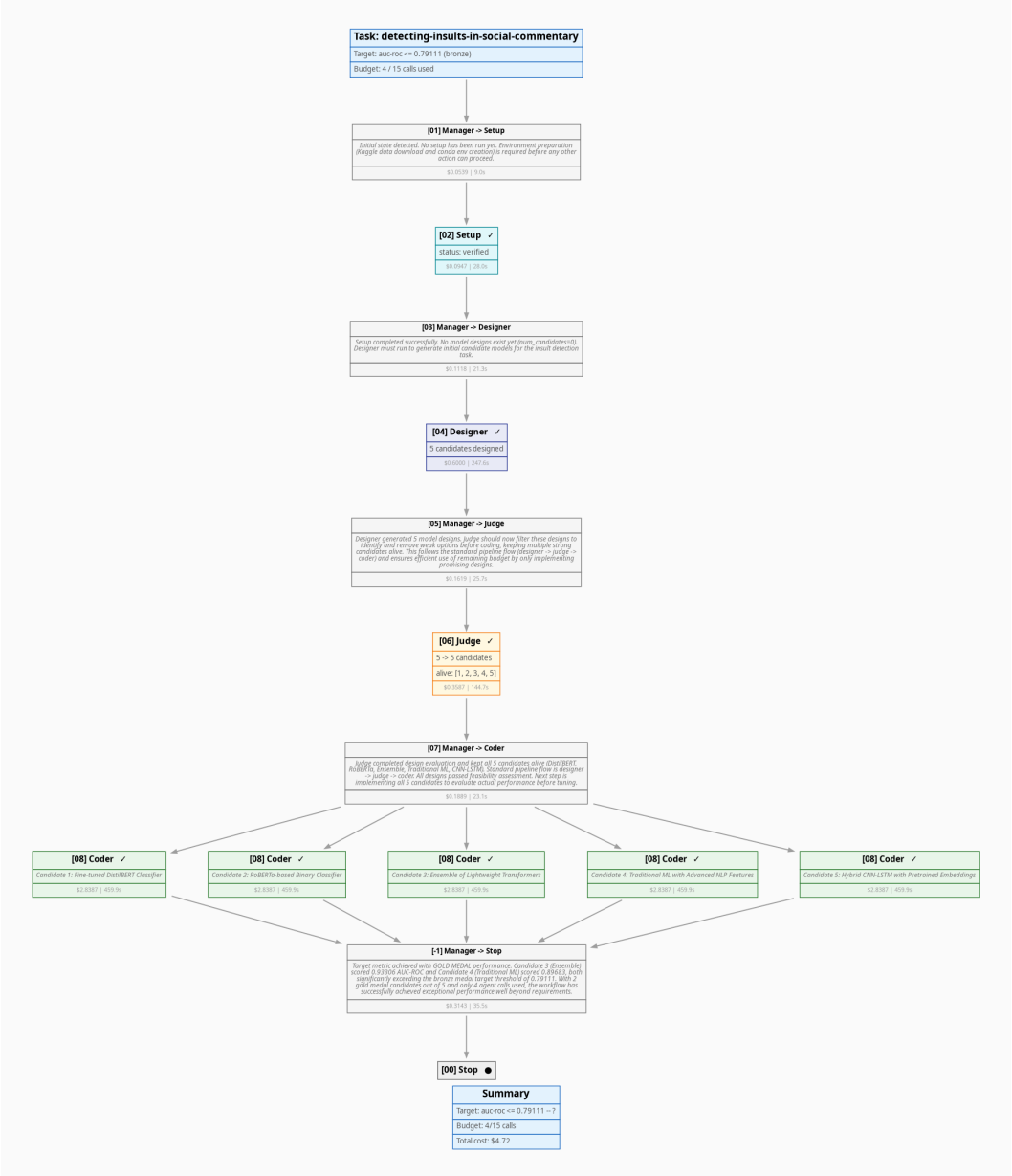

Figure 3: **Example manager-driven trajectory.** An example workflow trajectory from the *Detecting Insults in Social Commentary* task, illustrating how the manager orchestrates parallel execution and judging across five candidates. The workflow terminates once satisfactory performance is achieved, triggering final objective grading.

4 Discussion

The results demonstrate that a structured multi-agent workflow can be highly effective for solving end-to-end AI tasks, achieving gold-medal-level performance on multiple challenging Kaggle competitions. By decomposing the AI development process into specialized roles (e.g., design, implementation, evaluation, tuning, and revision) and enabling parallel exploration of candidate solutions, our system is able to iteratively refine models in a manner that closely resembles the process used by human AI researchers. Despite these promising outcomes, a key limitation of multi-agent systems is their potentially high computational and inference cost, since multiple agents may be invoked repeatedly across several refinement cycles. Future work should focus on improving efficiency

through better budget allocation, more selective agent invocation, and more effective management of context length, including compressing experimental history, organizing intermediate artifacts, and designing communication protocols that allow agents to share only the most relevant information needed for subsequent decisions.

References

[1] Anthropic. Building agents with the claude agent sdk, 2025. Official blog post about the Claude Agent SDK.

[2] Anthropic. Introducing claude sonnet 4.5, 2025. Large language model optimized for coding and agentic tasks.

[3] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Aleksander Madry, and Lilian Weng. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations*, 2025.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mo Bavarian, Clemens Winter, Phil Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, Suchir Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374, 2021.

[5] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: a survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, IJCAI '24, 2024.

[6] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. Aide: Ai-driven exploration in the space of code. 2025.

[7] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[8] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.